

TreeMap Interview Handbook

1. Why was TreeMap created?

HashMap provides very fast lookups but does not keep keys sorted. TreeMap was introduced for situations where data must always remain in sorted order while still supporting efficient search, insertion and deletion.

2. Real-World Examples

Leaderboards, student records sorted by roll number, bank accounts sorted by account number, calendars sorted by date, and dictionaries sorted alphabetically.

3. What is TreeMap?

TreeMap stores data as key-value pairs just like HashMap, but the keys are always kept in sorted order. Sorting is based on the natural ordering of keys or a custom Comparator.

4. Internal Working

TreeMap is implemented using a Red-Black Tree, which is a self-balancing Binary Search Tree. After every insertion or deletion, Java automatically balances the tree to keep operations efficient.

5. Why Red-Black Tree?

A normal Binary Search Tree can become skewed and degrade to $O(n)$. A Red-Black Tree keeps the height close to balanced, so searching, inserting and deleting remain $O(\log n)$.

6. put() Flow

Compare the key with the current node. Move left if smaller or right if larger. Insert the new node, then rebalance the tree if required.

7. get() Flow

Start at the root and compare the requested key. Move left or right until the matching key is found or the search ends.

8. Time Complexity

Operation	Complexity	Reason
put()	$O(\log n)$	Tree traversal + balancing
get()	$O(\log n)$	Balanced tree search
remove()	$O(\log n)$	Balanced tree update
firstKey()/lastKey()	$O(\log n)$	Traverse to leftmost/rightmost node

9. HashMap vs TreeMap

HashMap: Average $O(1)$, no ordering, uses hashing. TreeMap: $O(\log n)$, keeps keys sorted, uses a Red-Black Tree.

10. Comparator

If natural ordering is not suitable, pass a Comparator while creating the TreeMap to define custom sorting logic.

11. Common Interview Questions

Why is TreeMap slower than HashMap? Which data structure does TreeMap use? When should TreeMap be preferred over HashMap? Can TreeMap store null keys? (No, because keys must be comparable.)

12. Common Mistakes

Choosing TreeMap when ordering is not required, expecting $O(1)$ lookups like HashMap, and confusing TreeMap with a Binary Search Tree that is not self-balancing.

13. Remember Forever

TreeMap = Red-Black Tree. Keys are always sorted. put/get/remove = $O(\log n)$. Use TreeMap when sorted keys matter; use HashMap when maximum lookup speed is the priority.